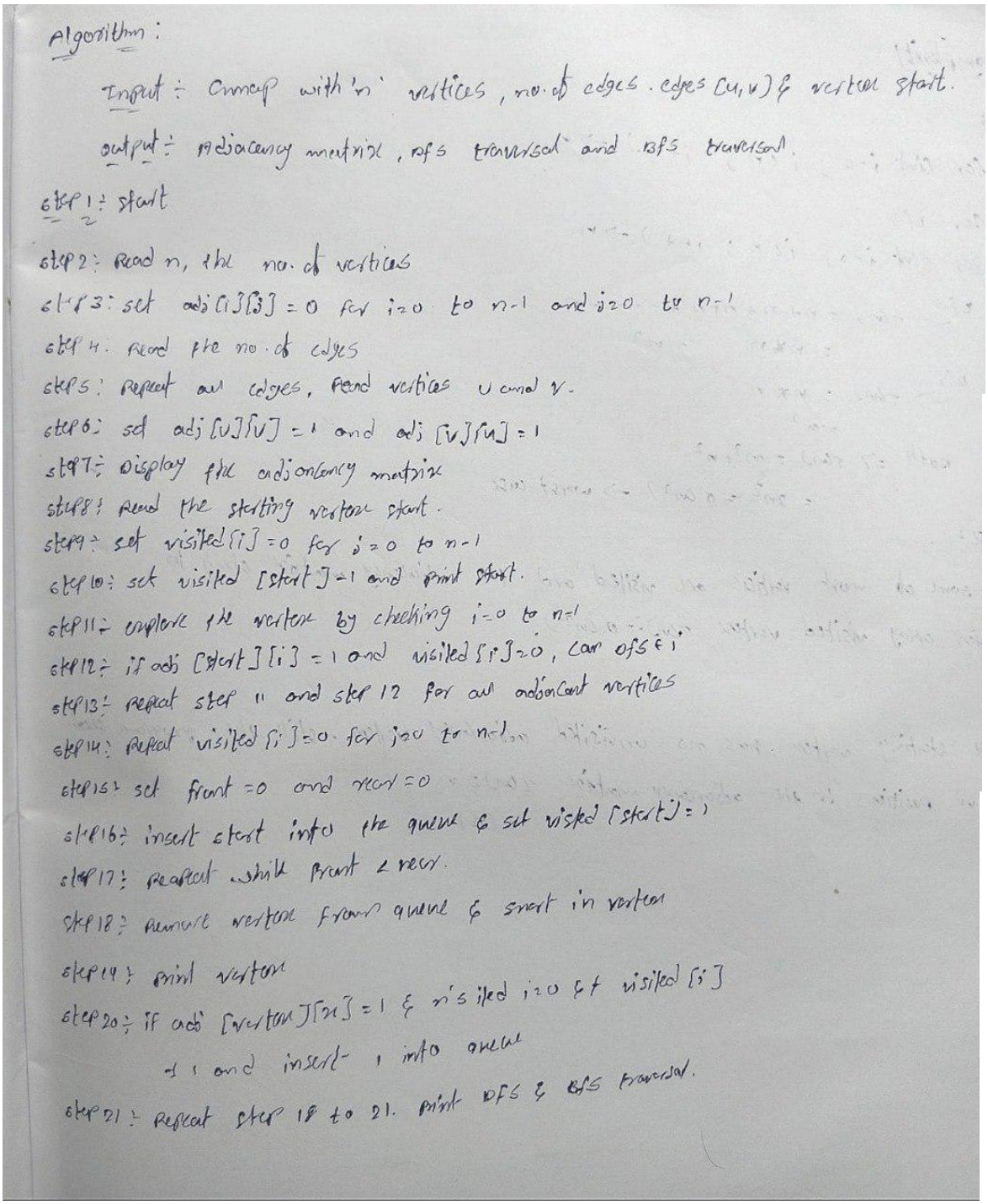
PRACTICAL-08

AIM : Implementation of Graph and Searching (DFS and BFS).

ALGORITHM :



Algorithm :

Input :- Create with 'n' vertices, no. of edges. edges (u,v) & vertex start.

output :- adjacency matrix, dfs traversal and bfs traversal.

step 1 :- start

step 2 :- Read n, the no. of vertices

step 3 :- set adj[i][j] = 0 for i=0 to n-1 and j=0 to n-1

step 4 :- Read the no. of edges

step 5 :- Repeat all edges, Read vertices u and v.

step 6 :- set adj[u][v] = 1 and adj[v][u] = 1

step 7 :- Display the adjacency matrix

step 8 :- Read the starting vertex start.

step 9 :- set visited[i] = 0 for i = 0 to n-1

step 10 :- set visited[start] = 1 and print start.

step 11 :- explore the vertex by checking i=0 to n-1

step 12 :- if adj[start][i] = 1 and visited[i] = 0, call dfs(i)

step 13 :- Repeat step 11 and step 12 for all adjacent vertices

step 14 :- Repeat visited[i] = 0 for i=0 to n-1

step 15 :- set front = 0 and rear = 0

step 16 :- insert start into the queue & set visited[start] = 1

step 17 :- Repeat while front < rear.

step 18 :- Remove vertex from queue & store in vertex

step 19 :- print vertex

step 20 :- if adj[vertex][x] = 1 & visited[i] = 0 & visited[i] = 1 and insert i into queue

step 21 :- Repeat step 18 to 21. Print DFS & BFS traversal.

PROGRAM :

```
#include <iostream>
using namespace std;
class Graph
{
public:
int adj[10][10];
int visited[10];
int n;
void getInput()
{
int edges;
cout << "Enter number of vertices: "; cin >> n;
for(int i = 0; i < n; i++)
{
for(int j = 0; j < n; j++)

{
adj[i][j] = 0;
}
}
cout << "Enter number of edges: ";
cin >> edges;
cout << "Enter edges:\n";
for(int i = 0; i < edges; i++)
{
int u, v;
cin >> u >> v;
adj[u][v] = 1;
adj[v][u] = 1;
}
}
void displayGraph() {
cout << "\nAdjacency Matrix:\n";
for(int i = 0; i < n; i++){
for(int j = 0; j < n; j++)
{
cout << adj[i][j] << " ";
}
cout << endl;
}
}
void DFS(int start)
```

```
{
visited[start] = 1;
cout << start << " ";
for(int i = 0; i < n; i++){
if(adj[start][i] == 1 && visited[i] == 0) {
DFS(i);
}
}
}
void BFS(int start)
{
int queue[10];
int front = 0;
int rear = 0;
for(int i = 0; i < n; i++) {
visited[i] = 0;
}

visited[start] = 1;

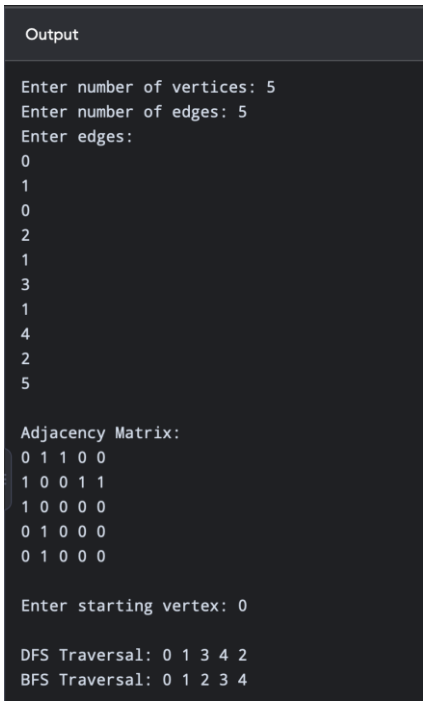
queue[rear] = start;

rear++;

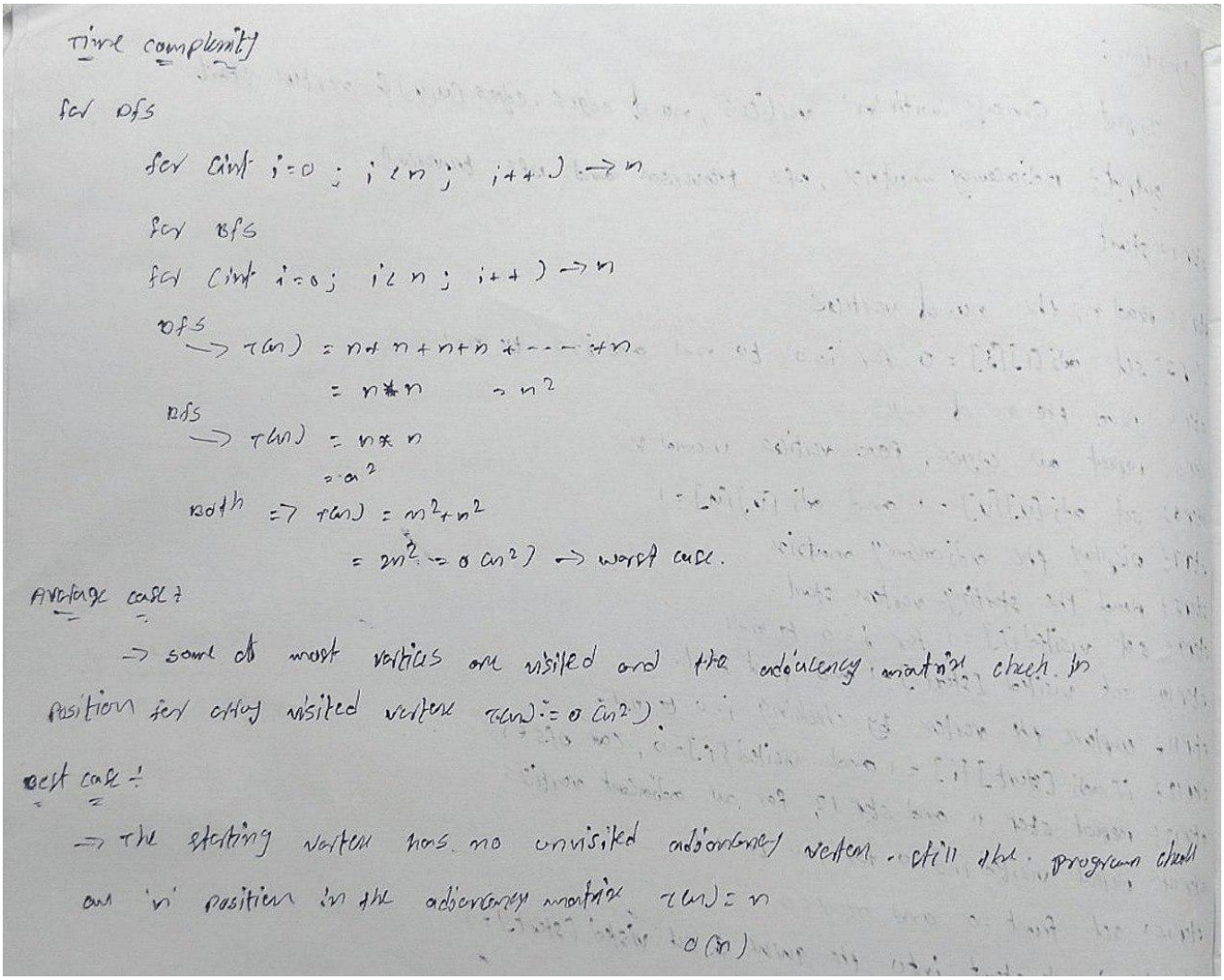
while(front < rear)
{
int vertex = queue[front];
front++;
cout << vertex << " ";
for(int i = 0; i < n; i++)
{
if(adj[vertex][i] == 1 && visited[i] == 0)
{
visited[i] = 1;
queue[rear] = i;
rear++; }
}
}
}
};
int main()
{
Graph g;
int start;
```

```
g.getInput();
g.displayGraph();
cout << "\nEnter starting vertex: ";
cin >> start;
for(int i = 0; i < g.n; i++)
{
g.visited[i] = 0;
}
cout << "\nDFS Traversal: ";
g.DFS(start);
cout << "\nBFS Traversal: ";
g.BFS(start);
cout << endl; return 0;
}
```

OUTPUT :



TIME COMPLEXITY :



CONCLUSION:

I implemented a Graph data structure and performed graph traversal using Depth First Search (DFS) and Breadth First Search (BFS). I represented the graph using an adjacency list and used a visited array to keep track of the vertices that had already been explored. In DFS, I explored each adjacent vertex deeply before backtracking, while in BFS, I used a queue to visit the vertices level by level. I tested both algorithms with different graph inputs and verified their traversal orders. Through this implementation, I understood the practical difference between DFS and BFS and learned how both algorithms can efficiently traverse a graph with a time complexity of O(V + E).